Ex 2 - redone

2. Git guide

a) B* b[] = {new B{}, new D{}};

B() → B{}   B() → B{}
D() → D{}

b[0] → print()
↳ type: B*
↳ real: B
↳ print() virtual in the base class
} => REAL => B => print => "B"

b[1] → print()
↳ type: B*
↳ real: B
virtual
} => REAL => D => print => "D"

delete b[0]

Screen: B{} B{} D{} B D ~B() ~D() ~B()
$\underbrace{\text{~B()}}_{\text{delete b[0]}}$ $\underbrace{\text{~D() ~B()}}_{\text{delete b[1]}}$

b) Person ~~*p~~ = new Person{};
delete p;
Person* s = new Student{}; ✓ → type pointer: Person* ; real: Student ; virtual
s → print();
delete s; → Student → ~Student() → ~Person()

Screen: ~~Person{} ~Person{}~~ Person{} Student{} Student ~Student() ~Person()

c) int res = 0
res = fct2(-5) → throw E{}
↳ E() => E{}

catch
↳ DE::m = 0

→ destructor ~E()

Screen: E{} 0 ~E()

d) B A

D ~~B~~ A C

after!

2. "row 2", adder

a) fct ("Hi")
fct ("Hello")

Screen: 101 Not empty

b)

v: | 1 | 2 | 3 | 4 | 5 | => | 1 | 2 | ~~3~~ | 7 | 4 | 5 |

0 1 2 3 4 ; 0 1 2 3 4 5

insert Before! ; 11

copy_if --- back_inserter (a%2==1) => 1 11 7 5.

c) B* b = new B{} → type: B* ; real: B

B* d = new D{} → type: B* ; real: D

d->f() virtual } => REAL => ~~D~~::f => B::f() => $\boxed{\text{"B.f"}}$ $\boxed{\text{"D.f"}}$

d->g(*b) not virtual } => TYPE: B* => B::g() => $\boxed{\text{B.g}}$

*b obj. => B() => $\boxed{\text{copy}}$

↳ $\boxed{\text{NB}}$

Screen: B.f D.f ~~B~~ copy B.g NB NB NB

↓ *b ↓ b ↓ d

d) A a1, a2; //a1->0
//a2->0

a1.set(8) //a1 -> 8
//a2->0

A a3 ; //a3 ->0

! a3=a1 ;//automatic assignment op.

A a4=a1; //copy constructor

a1.set(5)

a1 -> ~~8~~ 5 ← a3

a4 -> 8

=> 5 0 5 8

{ ∃ copy constr.
∃ ~~ass~~ destructor
∄ ass. op.

~A() — delete x => double delete => error.

2018-1

Screen: 1 10 40

except(true) → threw 10.

) B* b1 = new B{}; → type: B* → real: B

b1 → f(); } => real => B::f => $\boxed{\text{B.f}}$
virtual

B* b2 = new D{b1}; → t: B* → real: D

b2 → f() } => D::f => b.f() → $\boxed{\text{B.f}}$
$\boxed{\text{D.f}}$

delete b2; => $\boxed{\text{ND}}$ $\boxed{\text{NB}}$
delete b1; => $\boxed{\text{NB}}$

Screen:
B.f B.f D.f ND NB NB

c) answer to life 42

d)

a	b	~~c~~	~~d~~

b a

invalid iterator: { str.erase(it);
str. ~~erase~~ insert (it, "b");

↓

correct: { it = str.erase(it);
str.insert(it, "b");

=> a b b a

2.2018-2

a) B* b1 = new B{}; type: B*, real: B

b1->f() virtual } => real => B::f => $\boxed{\text{B.f}}$

already ∃ ← ref, not copy

B* b2 = new D{*b1}; type: B*, real: D

b2->f() virtual } => real => D::f => $\boxed{\text{D}}$

~~default constructor B~~ b.f() => $\boxed{\text{B.f}}$

delete b2; => $\boxed{\text{~D ~B}}$

delete b1 => $\boxed{\text{~B}}$

Screen: B.f D B.f ~D ~B ~B

B b; → new obj. (constr / destr.)

B& b; → only alias, no new obj.

B* b; → only pointer, no new obj.

b) Screen: One Positive Negative

except(3) → return Positive

except(-2) → throws Negative

c) fct2 (10, 10, 5, 5) => 10 10 10

$\underbrace{\text{int int}}_{T}$ $\underbrace{\text{int int}}_{U}$

fct2 (10, 10.5, 5, 5) => 10 10.5 5

$\underbrace{\text{10, 10.5}}_{\text{double}}$ $\underbrace{\text{5, 5}}_{\text{int}}$

fct2 (-2, -2, "Good", "luck!") => -2 -2 Good luck

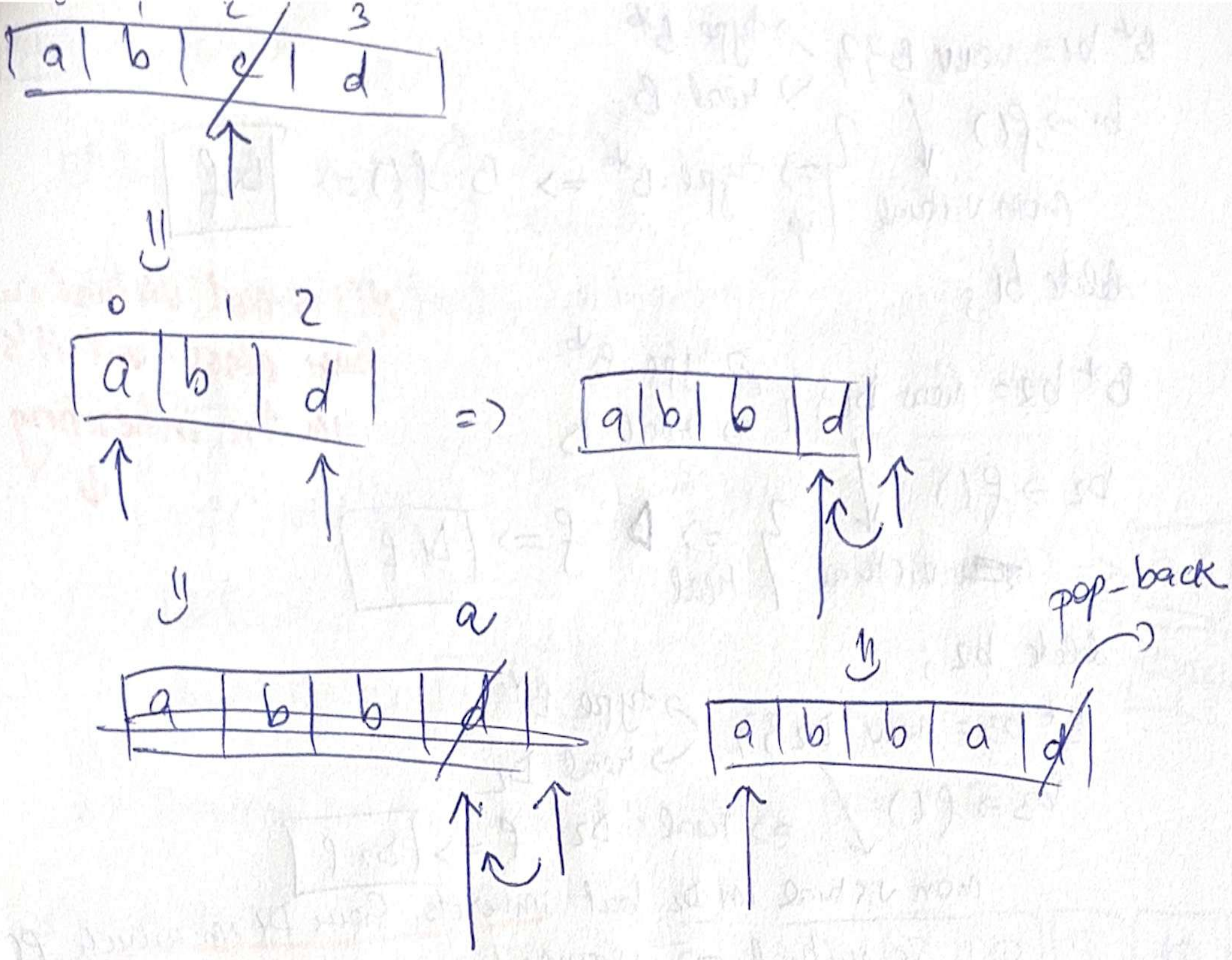
$\underbrace{\text{-2, -2}}_{\text{int}}$ $\underbrace{\text{"Good", "luck!"}}_{\text{string}}$

fct2 <int, A> (1, 1, A{2}, A{3}) => ~~1 1~~ => error. ∄

∄ operator<< for A

=> Screen: 10 10 10 10 10.5 5 -2 -2 Good luck

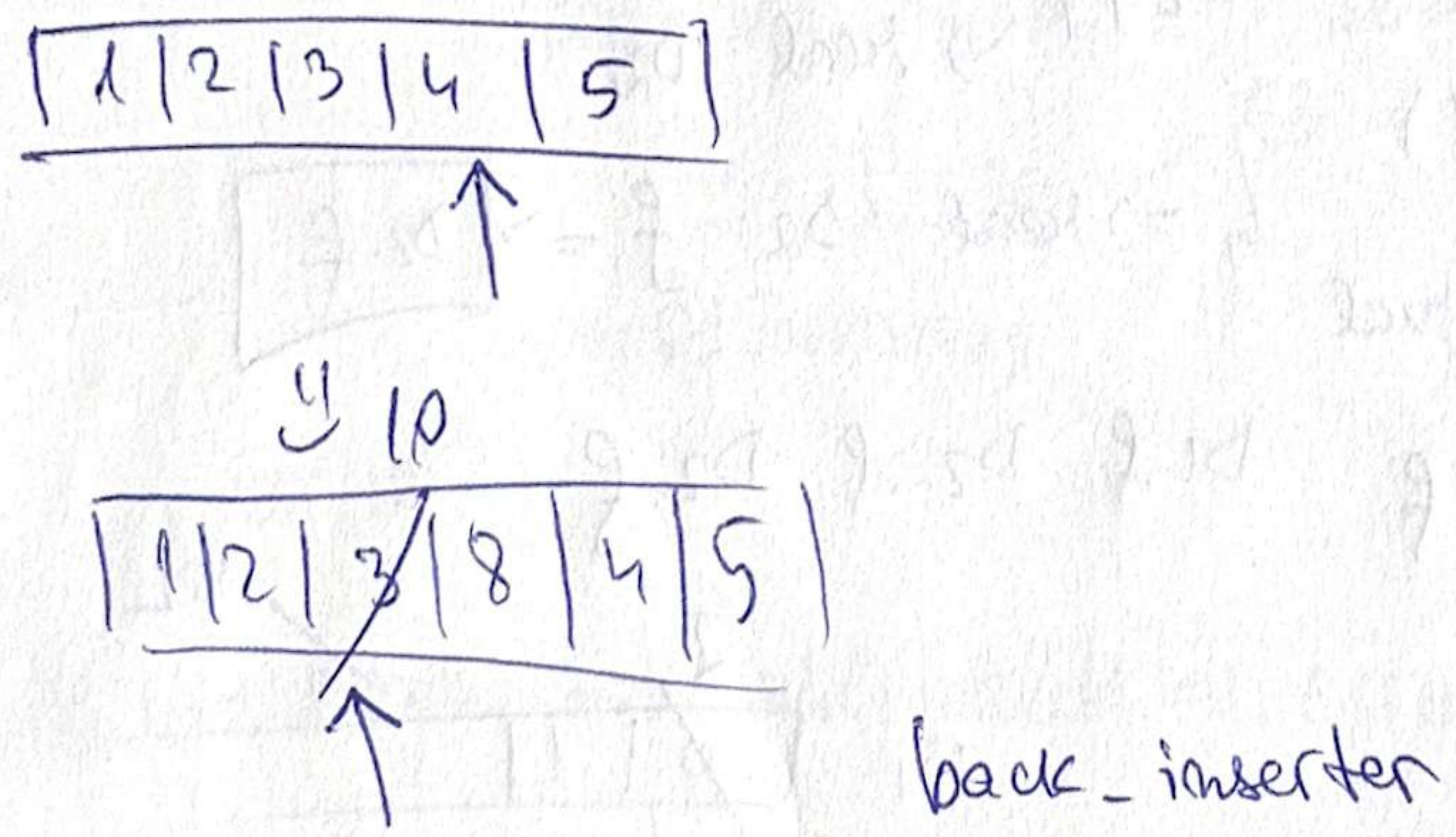
4



=> a b b a.

~~2. 3. jpg~~

a)



=> 2 10 8 4

b) Start Exception1 Exception2 Exception1 Copy_ex1

except(0) → throw Ex2{}

=> Exception1 Exception2

except(-2) => throw Ex1{}

=> Exception1

$\underbrace{\text{catch (Ex1 e)}}$

by value => copy constr of Ex1 => "Copy_ex1"

c) B* b1 = new B{}; ↗ type: B*
↘ real: B

b1 → f() ↓ } ⇒ type: B* ⇒ B::f() ⇒ $\boxed{B.f}$
non virtual

delete b1;

f() is not virtual in the base class but it's virtual in the inheriting class ↓

B* b2 = new D1{}; ↗ type: B*
↘ real: D

b2 → f() ↓ } ⇒ D::f ⇒ $\boxed{D1.f}$
virtual real

delete b2;

B* b3 = new D2{}; ↗ type: B*
↘ real: D2

b3 → f() ✓ ⇒ real: D2::f ⇒ $\boxed{D2.f}$

non virtual in D2 but inherits from D1 in which f() is virtual ⇒ VIRTUAL!

dellete b3.

D1* d = new D2{}; ↗ type: D1*
↘ real: D2

d → f() } ⇒ real: D2::f ⇒ $\boxed{D2.f}$
virtual

Screen: B.f D1.f D2.f D2.f

d) Vector v1;

v1.add(0)

v1.add(1)

Vector v2 = v1; ∄ copy constructor ⇒ shallow copy.
default

(pretend ∃) ⇒ v2 = v1 ...

v1[0] = 2

v1: | ~~0~~ 2 | 1 | ← v2

⇒ 2 1 2 1

+ goes out of scope ⇒ ~Vector() ...

⇒ deletes twice ~ dangling (v1, v2)

rule of three.
{ ∄ copy constr.
∃ destructor
∄ assignment op.

.jpeg

c) B* b = new B(); → type: B* → real: B

B* d = new D{6}; → type: B* → real: D ⇒ $\boxed{D}$

d→f()
f() virtual } ⇒ real: D::f() ⇒ $\boxed{D.f}$

b→f() ⇒ $\boxed{B.f}$

$\boxed{copy}$

delete d; ⇒ $\boxed{NB}$
delete b ⇒ $\boxed{NB}$

⇒ Screen: D D.f B.f copy NB NB

d) A a1, a2; // a1 = 5
a2 = 5
nr of Instances = 2 ⇒ $\boxed{2}$ $\boxed{2}$ $\boxed{10}$ $\boxed{5}$ $\boxed{10}$

A a3=a1; //no default copy constructor ⇒ shallow copy (copy the pointer) (it generates automatically a copy constructor that is used here, not the written one (so nr of Instances stays 2)

a1 → ~~5~~ 10 ← a3

nr Of Instances

a1. set(10)

! out of scope ⇒ double delete ⇒ error.

2. 4.2 jpeg

a) except(10<2) ⇒ $\boxed{0}$
except(-2) ⇒ $\boxed{1}$

Screen: 0 1 Done

b)
0	1	2	3	4	5
10	9	~~8~~	7	6	5

⇒
0	1	2	3	4
10	9	7	~~6~~	~~5~~

11 1

⇒
10	9	7	11	1

dequeue = remove + return elem. from the front
front_inserter() ⇒ inverse order of the odd nr. in the original vector ⇒ 1 11 7 9

c) except (5<5) => $\boxed{0}$

except (-2) => $\boxed{-}$

Screen: 0 - +

d)

0	1	2	3	4	5
0	2	~~4~~	6	8	10

=>

0	1	2	3	4
0	2	~~7~~	~~8~~	10

=> 0 2 0 10.

=> front_inserter => 10 0 2 0.

g(B* b)	VS g(B& b) ? ?
call: g(&obj) / g(ptr)	call: g(obj) / g(*ptr)
b→	b•
b can be made to point elsewhere	b stays to the orig. obj.
can be nullptr	cannot be null

2. 9/5 (2020)

a) B* b = new B{} → type: B*, ↳ real: B

B* d = new D{} → type: B*, ↳ real: D

d → f() virtual } => real => D::f() => { B::f() => $\boxed{B.f}$ ✓ ; $\boxed{D.f}$ ✓ }

d → g(*b) I. g not virtual } => type: B => B::g() => $\boxed{B.g}$ II.

→ copy constructor => $\boxed{\text{copy}}$ I.

g(B& b) → ORIGINAL (no copy ctr.) OBJECT

NEW OBJECT g(B b) →

=> Out of scope => destructor => $\boxed{\sim B}$ III

g(B* b) → ORIG. OBJECT (no copy ctr.)

delete b; => $\boxed{\sim B}$

delete d; => $\boxed{\sim B}$

Screen: B.f D.f copy B.g ~B ~B ~B

I. II. III.

b) A a1(2) // a1 = 2 ; count = 1

A a2(4) // a2 = 4 ; count = 2 $\boxed{2}$ endl → 2 11 4 11 11 3

a1.set(18) a1 → ~~18~~ 11 ← a3, ← a4

A a3 // a3 = 0 , count = 3

a3 = a1 // no ~~copy constructor~~ assignment operator => compiler generates one automatically

A a4 = a1; // no copy constr. => compiler generates one autom.

a1.set(11) ↓ default generated => shallow copy